

Manual de Usuario — BattleScript

Organización de Lenguajes y Compiladores 1 — Proyecto 1

Zabdi Merari Adina Donis Rosales – 202308487

Contenido

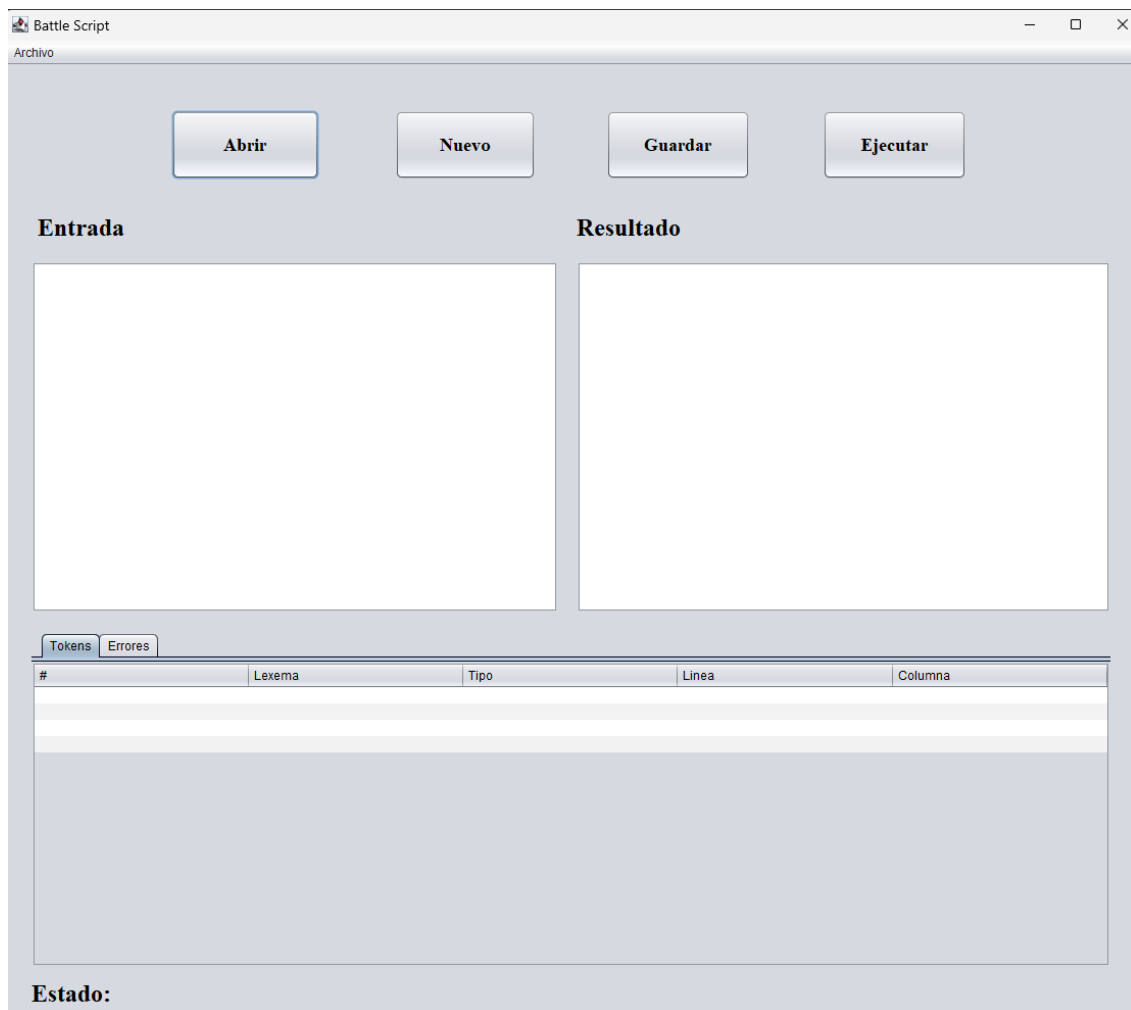
1. Introducción	1
2. Interfaz principal	1
3. Crear un archivo	3
4. Abrir	3
5. Guardar	3
6. Ejecutar	3
7. Tokens	4
8. Errores	4
9. Estructura de un programa	7
10. Ejemplo	7
11. Errores frecuentes	7
12. Cerrar la aplicación	8

1. Introducción

BattleScript es una aplicación de escritorio (Java, Swing) que permite escribir, analizar y ejecutar archivos con extensión `.bt1`. Cada archivo `.bt1` define estrategias de combate (magos y guerreros), partidas y un punto de entrada (`main`) que determina qué partidas se ejecutan y con qué semilla aleatoria. Este manual explica cómo usar el entorno de trabajo: crear, abrir y guardar archivos, ejecutar el análisis léxico/sintáctico, interpretar los reportes generados y leer los resultados de las batallas.

2. Interfaz principal

La ventana contiene un área de **Entrada** para el código fuente, un área de **Resultado** para las batallas, pestañas de **Tokens** y **Errores**, y una etiqueta de **Estado**. Los botones principales son **Abrir**, **Nuevo**, **Guardar** y **Ejecutar**.



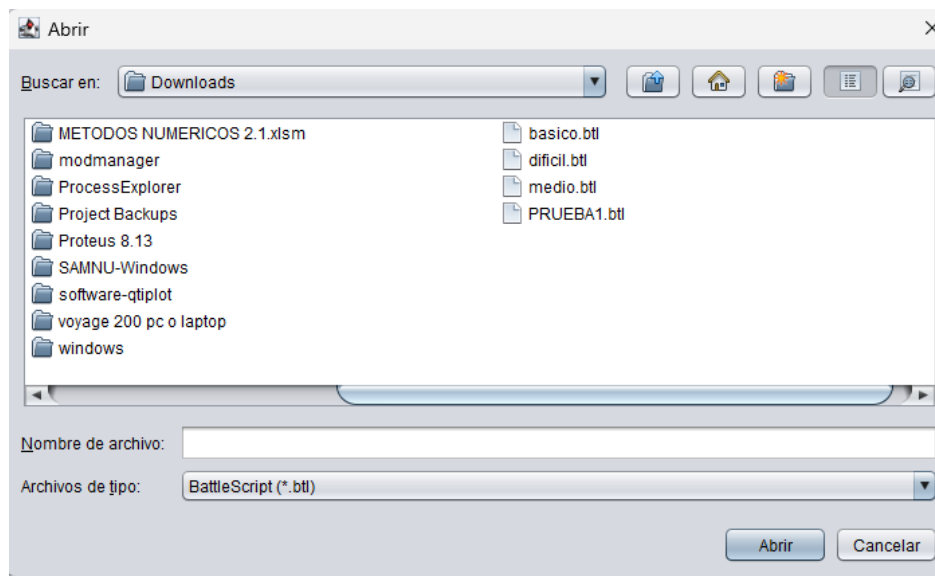
3. Crear un archivo

Presione **Nuevo** o use **Archivo** → **Nuevo**. Se limpia el editor y también se eliminan los tokens, errores y resultados anteriores.

La barra de estado muestra el mensaje “Nuevo archivo creado”. El archivo nuevo no queda asociado a ninguna ruta en disco hasta que se guarda por primera vez.

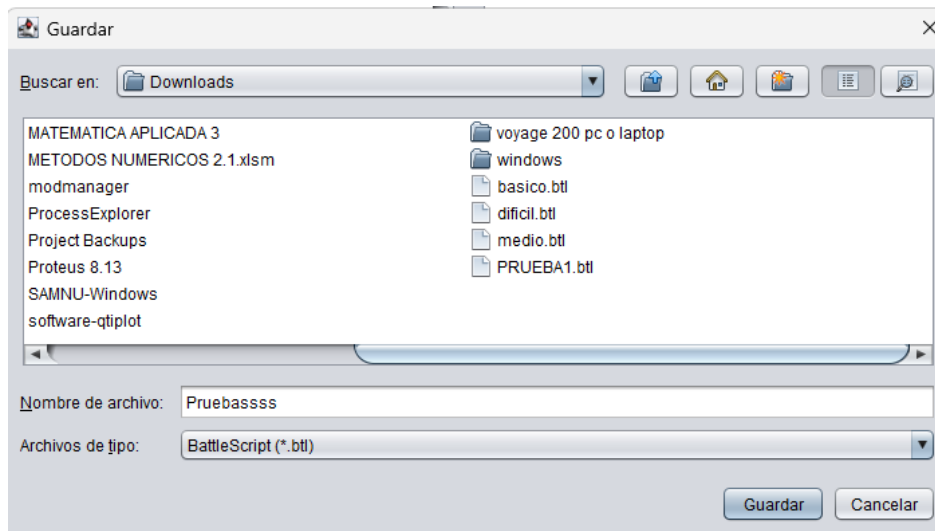
4. Abrir

Presione **Abrir** o **Archivo** → **Abrir**. Se muestra un selector de archivos filtrado para `.btl`. Al elegir un archivo, su contenido se carga en el editor.



5. Guardar

Presione **Guardar** o **Archivo** → **Guardar**. Si el archivo no tiene ruta, se solicita una ubicación. La aplicación agrega la extensión `.btl` si no fue escrita.

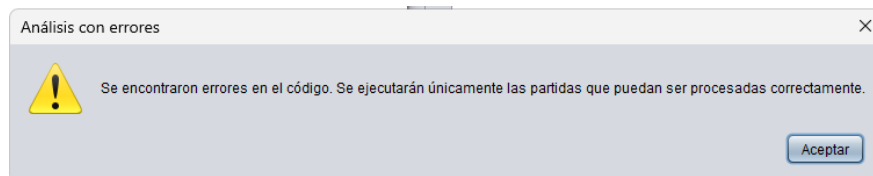


6. Ejecutar

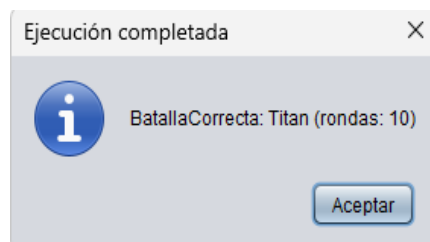
1. Escribir o cargar el código `.btl` en el editor.
2. Presionar el botón **Ejecutar**.
3. Si el editor está vacío, la aplicación muestra una advertencia y detiene la ejecución.

Al ejecutar, la aplicación realiza en orden:

1. Análisis léxico y sintáctico del código fuente.
2. Llenado de la pestaña **Tokens** con todos los tokens reconocidos.
3. Llenado de la pestaña **Errores** con los errores léxicos y sintácticos encontrados (si los hay).
4. Generación automática de un reporte HTML, que se guarda en la carpeta `reports/` del proyecto y se abre automáticamente en el navegador predeterminado.
5. Si se encontraron errores, la ejecución continúa únicamente con las partidas que puedan procesarse correctamente, y se muestra una ventana de advertencia.
6. Si el análisis fue exitoso, se ejecutan las partidas definidas en el bloque `main`, en el orden y con las semillas indicadas por cada instrucción `run`.
7. Se muestra una ventana emergente con el resumen de cada partida: nombre, ganador y cantidad de rondas jugadas.



Ventana de advertencia cuando el análisis encuentra errores



Ventana de resumen al completar la ejecución

Tras cerrar las ventanas emergentes, el editor, el área de resultado y las tablas de tokens y errores quedan visibles con la información de la última ejecución:

Entrada

```
mage DarkMage {
  initial: ARCANES_BOLT
  rules[
    if round_number >= 2
    then FIREBALL,
  ]
  else MEDITATE
}

warrior Titan {
  initial: SLASH
  rules[
    if self_resource <= 20
    then REST.
```

Resultado

```
~~~~~
Partida: BatallaCorrecta
Ganador: Titan
Rondas: 10
Log:
Titan usa SLASH causando 26 daño
DarkMage recupera 25 de recurso
Titan usa SLASH causando 26 daño
DarkMage recupera 25 de recurso
Titan usa SLASH causando 26 daño
DarkMage usa FIREBALL causando 42 daño
Titan usa SLASH causando 26 daño
Titan ha derrotado a DarkMage
```

Tokens Errores

#	Lexema	Tipo	Línea	Columna
1	mage	MAGE	1	1
2	DarkMage	ID	1	6
3	{	LLAVEA	1	15
4	initial	INITIAL	3	5
5	:	OJOS	3	12
6	ARCANE_BOLT	MAGE_ACTION	3	14
7	rules	RULES	5	5
8	:	OJOS	5	10
9	[	CORCHETEA	5	11
10	if	IF	6	9
11	round_number	ROUND_NUMBER	6	12
12	>=	GE	6	25
13	2	ENTERO	6	28
14	then	THEN	7	9
15	FIREBALL	MAGE_ACTION	7	14
16	,	COMA	7	22
17	else	ELSE	9	9

Estado: Ejecución completada - 1 partida(s)

7. Tokens

La tabla muestra número, lexema, tipo, línea y columna. Permite revisar qué reconoció el analizador léxico.

#	Lexema	Tipo	Línea	Columna
1	mage	MAGE	1	1
2	DarkMage	ID	1	6
3	{	LLAVEA	1	15
4	initial	INITIAL	3	5
5	:	OJOS	3	12
6	ARCANE_BOLT	MAGE_ACTION	3	14
7	rules	RULES	5	5
8	:	OJOS	5	10
9	[	CORCHETEA	5	11
10	if	IF	6	9
11	round_number	ROUND_NUMBER	6	12
12	>=	GE	6	25
13	2	ENTERO	6	28
14	then	THEN	7	9
15	FIREBALL	MAGE_ACTION	7	14
16	,	COMA	7	22
17	else	ELSE	9	9

8. Errores

La tabla muestra número, descripción, tipo, línea y columna. Sirve para localizar errores léxicos, sintácticos y semánticos

#	Descripción	Tipo	Línea	Columna
1	Token no reconocido (token: '\$' tipo: Léxico)	Léxico	68	5
2	Token no reconocido (token: '\$' tipo: Léxico)	Léxico	68	6
3	Token no reconocido (token: '\$' tipo: Léxico)	Léxico	68	7
4	Token no reconocido (token: '@' tipo: Léxico)	Léxico	69	5
5	Token no reconocido (token: '@' tipo: Léxico)	Léxico	69	6
6	Token no reconocido (token: '@' tipo: Léxico)	Léxico	69	7
7	Token no reconocido (token: '?' tipo: Léxico)	Léxico	70	5
8	Token no reconocido (token: '?' tipo: Léxico)	Léxico	70	6
9	Token no reconocido (token: '?' tipo: Léxico)	Léxico	70	7

9. Estructura de un programa

El programa contiene estrategias, partidas y el bloque `main`. Una estrategia define su clase, acción inicial y reglas. Una partida define jugadores, rondas, `scoring` y bonificaciones. `main` determina qué partidas se ejecutan y con qué `seed`.

Ejemplo

```
mage Merlin {
  initial: ARCANES_BOLT
  rules: [
    if self_resource <= 20 then MEDITATE,
    else ARCANES_BOLT
  ]
}

warrior Ragnar {
  initial: SLASH
  rules: [
    if self_resource <= 20 then REST,
    else SLASH
  ]
}

match DueloInicial {
  players: [Merlin, Ragnar]
  rounds: 5
  scoring: {
    damage_point: 1,
    healing_point: 1,
    successful_defense: 20,
    victory_bonus: 100,
    failed_action_penalty: 10
  }
}

main {
  run [DueloInicial] with {
    seed: 42
  }
}
```

Al ejecutarlo, el área de **Resultado** muestra el resumen y el registro de la partida:

```
Resultado

-----
Partida: BatallaCorrecta
Ganador: Titan
Rondas: 10
Log:
Titan usa SLASH causando 26 daño
DarkMage recupera 25 de recurso
Titan usa SLASH causando 26 daño
DarkMage recupera 25 de recurso
Titan usa SLASH causando 26 daño
DarkMage usa FIREBALL causando 42 daño
Titan usa SLASH causando 26 daño
Titan ha derrotado a DarkMage
```

10. Errores frecuentes

Verifique que las acciones correspondan a la clase de la estrategia, que las referencias a estrategias y partidas existan, que `scoring` tenga valores válidos y que la semilla y las rondas cumplan las restricciones. Los errores léxicos corresponden a símbolos o palabras que el lenguaje no reconoce.

Problema	Causa
Error léxico: carácter no reconocido	Se usó un símbolo que no pertenece al lenguaje (por ejemplo \$, #, @).
Error léxico: comentario sin cerrar	Se abrió un comentario <code>/*</code> y no se cerró con <code>*/</code> antes de terminar el archivo.
Error sintáctico: token inesperado	Falta una coma, una llave <code>{/}</code> , un corchete <code>[/]</code> , o el bloque <code>else</code> no está al final de las reglas.
Error semántico: acción no corresponde a la clase	Se usó, por ejemplo, <code>HEAVY_STRIKE</code> (acción de guerrero) dentro de una estrategia <code>mage</code> .
Error semántico: partida referenciada no existe	El bloque <code>main</code> hace <code>run [Nombre]</code> sobre una partida no definida.
Error semántico: semilla inválida	Se usó <code>seed: 0</code> , un número negativo o un decimal.

11. Cerrar la aplicación

Para cerrar la aplicación se puede usar **Archivo** → **Salir**, o el botón de cierre de la ventana.

No se solicita confirmación ni se guarda automáticamente el archivo abierto, por lo que se recomienda guardar antes de salir.